

Esquema do Banco de Dados Vetorial

Sistema Fluxora — Memória Semântica do Assistente de IA (pgvector)

1. Visão Geral

O banco de dados vetorial do Fluxora é implementado por meio da extensão pgvector, instalada na mesma instância PostgreSQL (Google Cloud SQL) utilizada pelo banco relacional. Diferentemente das tabelas tradicionais, este componente é responsável por armazenar representações numéricas (embeddings) de informações textuais, permitindo que o assistente de IA realize buscas por similaridade semântica.

Essa estrutura constitui a memória de longo prazo do assistente: trechos relevantes de conversas, transações, metas e preferências do usuário são convertidos em vetores e armazenados para consultas futuras, possibilitando respostas mais contextualizadas e personalizadas.

O esquema vetorial é composto por uma única tabela, `memory_embeddings`, complementada por dois índices especializados, um modelo de geração de embeddings externo e uma integração direta com o esquema relacional.

2. Tabela `memory_embeddings`

Armazena os registros de memória semântica do assistente. Cada linha representa um fragmento de informação (uma transação, uma preferência, um hábito identificado, entre outros) associado a um vetor numérico que codifica seu significado semântico.

Restrições:

- `user_id` referencia `users(id)`, da tabela do esquema relacional, com `ON DELETE CASCADE` — ao excluir um usuário, todas as suas memórias são removidas.
- O campo `type` aceita apenas os valores: `transacao`, `perfil`, `meta`, `habito`, `preferencia`, `outro`.
- O campo `embedding` é do tipo `vector(768)`, compatível com a dimensionalidade do modelo de geração utilizado.
- `reference_id` é uma referência opcional (sem FK formal) a um registro de origem no esquema relacional, como uma transação, meta ou conta.

Atributos — `memory_embeddings`

Atributo	Tipo	Restrição	Descrição
<u>id</u>	UUID	PK, NOT NULL	Identificador único do registro de memória
user_id	UUID	FK → users, NOT NULL	Referência ao usuário dono da memória
type	TEXT	NOT NULL, CHECK	Categoria: <code>transacao</code> <code>perfil</code> <code>meta</code> <code>habito</code> <code>preferencia</code> <code>outro</code>
content	TEXT	NOT NULL	Conteúdo textual original que

Atributo	Tipo	Restrição	Descrição
			originou o vetor
embedding	vector(768)	nullable	Representação vetorial semântica do conteúdo
reference_id	UUID	opcional	Referência opcional a registro relacional de origem
created_at	TIMESTAMPTZ	NOT NULL	Data e hora de criação do registro

3. Índices

Dois índices garantem o desempenho das consultas mais frequentes sobre a tabela `memory_embeddings`: um índice convencional para filtragem por usuário, e um índice especializado para busca por similaridade vetorial.

Índice	Coluna(s)	Tipo	Finalidade
idx_memory_user_id	user_id	B-Tree	Filtragem rápida de memórias por usuário
idx_memory_embedding	embedding	HNSW (cosine)	Busca por similaridade semântica entre vetores

O índice HNSW (Hierarchical Navigable Small World) é uma estrutura aproximada para busca do vizinho mais próximo, otimizada para alta dimensionalidade. A métrica de distância utilizada é a similaridade do cosseno, adequada para comparar a orientação semântica entre vetores de embedding.

4. Tipos de Memória

O campo `type` classifica cada registro armazenado em uma das seguintes categorias, permitindo que o assistente filtre e priorize o tipo de contexto recuperado durante uma conversa:

- `transacao` — fragmentos relacionados a movimentações financeiras registradas.
- `perfil` — informações sobre o perfil e características do usuário.
- `meta` — contexto relacionado a metas orçamentárias definidas.
- `habito` — padrões de comportamento financeiro identificados ao longo do tempo.
- `preferencia` — preferências expressas pelo usuário durante o uso do assistente.
- `outro` — categoria padrão para conteúdos que não se enquadram nas anteriores.

5. Modelo de Geração de Embeddings

A conversão de texto em vetor numérico é realizada por um modelo externo da Google, integrado ao backend da aplicação. O mesmo modelo é responsável tanto pela geração dos vetores armazenados quanto pela geração do vetor de consulta no momento da busca.

Característica	Valor
Modelo	text-embedding-004
Provedor	Google Gemini
Dimensionalidade	768
Métrica de distância	Cosseno (cosine similarity)

6. Integração com o Banco Relacional

Embora utilize um paradigma distinto de armazenamento, a tabela `memory_embeddings` coexiste na mesma instância PostgreSQL do esquema relacional e mantém um vínculo direto com ele:

- `user_id` é uma chave estrangeira formal para `users(id)`, garantindo que toda memória pertença a um usuário válido e seja removida em cascata caso o usuário seja excluído.
- `reference_id` é uma referência opcional, sem restrição de chave estrangeira, que pode apontar para registros de `transactions`, `budget_goals` ou `accounts`, permitindo rastrear a origem relacional de uma memória.

A consulta típica de recuperação de contexto utiliza o operador de distância do `pgvector` para ordenar os registros mais semanticamente próximos a um vetor de consulta:

```
SELECT content, type, reference_id
FROM memory_embeddings
WHERE user_id = $1
ORDER BY embedding <=> $2
LIMIT 5;
```

Nessa consulta, o operador `<=>` calcula a distância de cosseno entre o vetor armazenado e o vetor de consulta gerado a partir da mensagem atual do usuário, retornando os fragmentos de memória mais relevantes para compor o contexto enviado ao modelo de linguagem.

7. Observações Técnicas

- O banco vetorial não introduz uma nova instância de banco de dados: trata-se de uma extensão (`pgvector`) habilitada sobre a mesma instância PostgreSQL do esquema relacional.
- Por se tratar de uma única tabela sem relacionamentos N:N com outras entidades vetoriais, o esquema não é representado por um diagrama ER tradicional, mas por um diagrama estrutural que evidencia campos, índices e fluxo de geração dos vetores.
- O campo `embedding` é nullable para permitir o registro de conteúdo textual mesmo em casos excepcionais de falha na chamada ao modelo de geração de embeddings.

- A escolha do índice HNSW em detrimento de IVFFlat se justifica pelo melhor desempenho em buscas aproximadas de alta precisão, ao custo de maior uso de memória.